

Methods Section German

Methoden

Das Modell setzt an mit dem Bulk-Anteil des Energie-Impuls-Tensors auf der Brane in der Form $\mathcal{T}(\mathrm{Bulk}, \mu, \nu) = S(\rho) \Lambda(\mu, \nu)$. Hierbei ist $(S(\rho) = S_0 \rho^{-\eta})$ eine skalare Dichte-Skalierungsfunktion und $\Lambda(\mu, \nu)$ ein symmetrischer Struktur-Tensor. Für die Herleitung verwenden wir eine 1+3-Zerlegung $\Lambda(\mu, \nu) = \alpha u^\mu u^\nu + \beta h(\mu, \nu) + \Pi(\mu, \nu)$ mit (u^μ) als zeitartigem Viererfeld und dem Projektor $h(\mu, \nu) = g(\mu, \nu) + u^\mu u_\nu$.

Ausgangspunkt ist die kovariante Divergenz

$$\nabla_\nu \mathcal{T}(\mathrm{Bulk}, \mu, \nu) = S(\rho) \left[\partial_\mu \ln S + \Lambda(\mu, \nu) + \nabla_\mu \Lambda(\mu, \nu) \right],$$

die komponentenweise ausgewertet wird. Für das Spezialmodell $\Lambda(\mu, \nu) = \beta(r) h(\mu, \nu)$ in stationärer, sphärisch symmetrischer Geometrie (Schwarzschild-Nähe, Newton-Limit $(|\Phi| \ll 1)$) folgt in führender Ordnung für die radiale Komponente

$$Q_r \simeq S(\rho) \left[\beta'(r) - \eta \beta(r) \frac{d \ln \rho}{dr} + \frac{2\beta(r)}{r} \right]$$

mit kleinen Korrekturen proportional zu (Φ) und (Φ^2) . Die nicht-gravitative Beschleunigung eines Testkörpers mit mittlerer Dichte (ρ_m) ergibt sich dann als

$$a(\mathrm{NG})(r) \simeq \frac{S_0 \rho(r)^{-\eta}}{\rho_m} \left[\beta'(r) - \eta \beta(r) \frac{d \ln \rho}{dr} + \frac{2\beta(r)}{r} \right].$$

Zur Sicherstellung von Kausalität und Stabilität wird ein dynamisches Feldmodell für $\Lambda(\mu, \nu)$ vorgeschlagen, beispielhaft $\Box \Lambda(\mu, \nu) + m^2 \Lambda(\mu, \nu) + \gamma \nabla_\mu \nabla^\mu \Lambda(\mu, \nu) = \mathcal{S}(\mu, \nu)$, mit Bedingungen $(m^2 \geq 0)$ und geeigneter Wahl von (γ) für Hyperbolizität.

Numerische Validierung erfolgt in zwei Schritten: (i) symbolische Ableitung und Taylor-Expansion in kleinen Gravitationspotenzialen mit SymPy zur Kontrolle aller Indexterme, (ii) numerische Evaluierung und Inferenz mit parametrisierten Profilen $(\rho(r))$ (z. B. NFW oder r^{-2}) und $(\beta(r) = \beta_0 (1 + r/r_s)^{-p})$. Parameterinferenz wird mit MCMC (emcee) durchgeführt; Posterior-Summaries, N_{eff} und R_{hat} werden für Veröffentlichungsangaben dokumentiert.

Methods Section English

Methods

We consider the bulk contribution to the brane energy-momentum tensor as

$\mathcal{T}_{\mathrm{Bulk},\mu\nu} = S(\rho)\Lambda_{\mu\nu}$ where $(S(\rho) = S_0\rho^{-\eta})$ is a scalar density scaling function and $(\Lambda_{\mu\nu})$ is a symmetric structural tensor. We perform a 1+3 decomposition $(\Lambda_{\mu\nu} = \alpha u_\mu u_\nu + \beta h_{\mu\nu} + \Pi_{\mu\nu})$ with (u^μ) the timelike four-vector and $(h_{\mu\nu} = g_{\mu\nu} + u_\mu u_\nu)$ the projector.

Starting point is the covariant divergence

$$\nabla^\mu \mathcal{T}_{\mathrm{Bulk},\mu\nu} = S(\rho) \left[\partial^\mu \ln S \right] \Lambda_{\mu\nu} + \nabla^\mu \Lambda_{\mu\nu} S(\rho),$$

which is evaluated componentwise. For the special ansatz

$(\Lambda_{\mu\nu} = \beta(r)h_{\mu\nu})$ in a stationary, spherically symmetric geometry (Schwarzschild limit, Newton approximation $(|\Phi| \ll 1)$) the leading order radial component becomes

$$Q_r \simeq S(\rho) \left[\beta'(r) - \eta \beta(r) \frac{d \ln \rho}{dr} + \frac{2\beta(r)}{r} \right],$$

with small (Φ) corrections. The non-gravitational acceleration of a test mass with internal density (ρ_m) follows

$$a_{\mathrm{NG}}(r) \simeq \frac{S_0 \rho(r)^{-\eta}}{\rho_m} \left[\beta'(r) - \eta \beta(r) \frac{d \ln \rho}{dr} + \frac{2\beta(r)}{r} \right].$$

Causality and stability are enforced by proposing a dynamical field equation for

$(\Lambda_{\mu\nu})$ such as

$(\Box \Lambda_{\mu\nu} + m \Lambda^2 \Lambda_{\mu\nu} + \dots = \mathcal{S}_{\mu\nu})$ with $(m \Lambda^2 \geq 0)$ to ensure hyperbolicity. Validation proceeds via symbolic algebra

(SymPy) for index bookkeeping and Taylor series in (G) , followed by numerical evaluation and MCMC inference (emcee) for parameters (S_0, η, β_0, p) . Final reporting includes posterior medians, 1σ intervals, R-hat and effective sample sizes.

README with step-by-step reproduction commands

Reproduction quickstart

Prerequisites: conda, Python 3.8+ in activated environment, packages sympy numpy scipy emcee corner matplotlib arviz getdist.

1. Create and activate environment

- conda create -n fktenv python=3.10
- conda activate fktenv
- pip install sympy numpy scipy emcee corner matplotlib arviz getdist

2. Files (place in one folder)

- fktv42finalrun.yaml
- summarize_mcmc.py
- monitor_cobaya.sh (Linux/WSL) or PowerShell monitoring loop for Windows
- divergencelambdasymPy_full.py
- fitfktmcmc.py

3. Symbolic verification (SymPy script)

- python divergencelambdasymPy_full.py
- Output: symbolic expressions for Q_{nu} , Newton vs full, numeric sample evaluations
- If lambdify fails, script falls back to `.subs(...).evalf()` path.

4. Generate synthetic data and run MCMC fit

- python fitfktmcmc.py
- Output: mcmcchain.npy, cornerplot.png, bestfit_report.txt

5. Run Cobaya analysis (optional, for cosmological fits)

- Ensure conda env has cobaya and camb installed: `pip install cobaya camb mpi4py`
- Run Cobaya: `cobaya-run fktv42finalrun.yaml --allow-changes`
- Monitor with summarize_mcmc.py:
 - Linux/WSL: `./monitorcobaya.sh fktv42finalrunoutput 120`
 - Windows PowerShell:
`while ($true) { python .\summarizemcmc.py --chainsdir .\fktv42finalrunoutput --warnrhat 1.01 --warn_neff 10000; Start-Sleep -s 120 }`

6. Postprocessing and publication assets

- After MCMC: run summarize_mcmc.py to extract medians, σ , CI and LaTeX row:
 - `python summarizemcmc.py --chainsdir fktv42finalrun_output`
- Produce figures: cornerplot.png from fitfkt_mcmc.py; additional posterior predictive plots as needed.
- Create finalsummary.json containing:
 - modelid, rundate, environment (conda env pip freeze), chainfiles list, Neff per param, R_{hat} per param, median $\pm 1\sigma$ for key params.

7. Hash and archive reproducibility pack

- `sha256sum .yaml .py *.pdf > reproducibility_hashes.sha256`
- `zip -r repropack.zip fktv42finalrun.yaml summarizemcmc.py divergencelambdasymPyfull.py fitfktmcmc.py reproducibilityhashes.sha256`

Repro checklist and recommended parameter defaults

Repro checklist

- [] Conda environment created and activated (name fktenv)
- [] sympy, numpy, scipy, emcee, corner, matplotlib, arviz, getdist installed
- [] divergencelambdasymPy_full.py runs without obvious index inconsistencies

- [] fitfktmcmc.py produces MCMC chains and corner_plot.png
- [] Cobaya run (if used) produces chains in fktv42finalrun_output
- [] summarizemcmc.py extracts medians, $R_{\text{hat}} \leq 1.01$ and $N_{\text{eff}} \geq 10000$ or documents deviations
- [] finalsummary.json created with full metadata and SHA256 hashes stored

Recommended numeric defaults for example runs

- $S_0 \sim 1e-15$, $\eta = 1.0$, $\beta_0 = 0.1$, $p = 0.5$ for exploratory tests
- r grid: [1 AU, 10 AU, 100 AU, 1 kAU, 10 kpc] for sample outputs
- MCMC: nwalkers = 64, nsteps = 3000, burn_in = 500 for prototype; increase for production

PowerShell-Monitor (ready to run)

Speichere den folgenden Inhalt als Datei monitorcobaya.ps1 im selben Ordner wie summarizemcmc.py und führe ihn in einer PowerShell mit aktiviertem Conda-Environment aus.

```
`powershell
```

```
<#
```

```
monitor_cobaya.ps1
```

```
Usage examples:
```

```
default: folder .\fktv42finalrun_output, interval 120s
```

```
.\monitor_cobaya.ps1
```

```
custom:
```

```
.\monitorcobaya.ps1 -ChainsDir .\myoutput -IntervalSec 60 -WarnRhat 1.01 -WarnNeff 10000
```

Requirements:

- PowerShell (Windows) or pwsh (cross-platform PowerShell)
- Python available in PATH (use conda activate cobaya first if needed)
- summarize_mcmc.py present and runnable

```
>
```

```
param(
```

```
    [string]$ChainsDir = ".\fktv42finalrun_output",
```

```
    [int]$IntervalSec = 120,
```

```
    [double]$WarnRhat = 1.01,
```

```
    [int]$WarnNeff = 10000
```

```
)
```

Resolve full paths

```
$ScriptDir = Split-Path -Parent $MyInvocation.MyCommand.Definition
```

```
$Summarizer = Join-Path $ScriptDir "summarize_mcmc.py"
```

```

if (-not (Test-Path $Summarizer)) {
    Write-Error "summarize_mcmc.py not found in script folder: $ScriptDir"
    exit 2
}

Write-Host "Monitoring Cobaya output folder:" (Resolve-Path $ChainsDir) "interval:"
$IntervalSec "s"
Write-Host "Summarizer:" $Summarizer
Write-Host "Press Ctrl-C to stop.`n"

try {
    while ($true) {
        $ts = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
        Write-Host "=== [$ts] Checking chains in $ChainsDir ==="
        & python $Summarizer --chainsdir $ChainsDir --warnrhat $WarnRhat --warnneff
$WarnNeff 2>&1 | ForEach-Object { Write-Host "  $" }
        Write-Host ""
        Start-Sleep -Seconds $IntervalSec
    }
}
catch [System.Exception] {
    Write-Error "Monitor stopped due to error: $($_.Exception.Message)"
    exit 3
}
`

```

Kurz-Anleitung zur Nutzung

- Öffne Anaconda Prompt oder PowerShell, aktiviere dein Environment:
conda activate cobaya
- Wechsle in das Verzeichnis mit summarizemcmc.py und monitorcobaya.ps1.
- Starte das Script (PowerShell):
.\monitor_cobaya.ps1
- Oder mit eigenen Parametern:
.\monitorcobaya.ps1 -ChainsDir .\fktv42finalrunoutput -IntervalSec 60 -WarnRhat 1.01
-WarnNeff 10000

Das Script ruft periodisch summarize_mcmc.py auf, zeigt dessen Ausgabe eingerückt an und gibt Exit-Codes/Warnungen weiter. Abbruch mit Strg+C.